

Tutorial1: Basic Programming Tasks of OCaml

1 练习题1: Fibonacci function

```
# let rec fib n =  
  if n < 0 then failwith "invalid input"  
  else  
    match n with  
    | 0 -> 0  
    | 1 -> 1  
    | n -> fib (n - 1) + fib (n - 2) ;;  
val fib : int -> int = <fun>  
# fib 19 ;;  
- : int = 4181  
# fib (-3) ;;  
Exception: (Failure "invalid input")
```

2 练习题2: 列表累乘

```
# let prod lst = (*a tail recursive wrapped version*)  
  let rec inner_prod lst cal =  
    match lst with  
    | [] -> cal  
    | head :: tail -> inner_prod tail cal * head  
  in inner_prod lst 1 ;;  
val prod : int list -> int = <fun>  
# prod [2; 3; 5] ;;  
- : int = 30
```

3 练习题3: 复合函数

```
# let compose1 func2 func1 x =  
  let mid = func1 x  
  in func2 mid ;;  
val compose1 : ('a -> 'b) -> ('c -> 'a) -> 'c -> 'b = <fun>  
# let compose2 =  
  fun func2 func1 x -> func2 (func1 x) ;;  
val compose2 : ('a -> 'b) -> ('c -> 'a) -> 'c -> 'b = <fun>
```

```
# let add_one x = x + 1 ;;
val add_one : int -> int = <fun>
# let double x = x * 2 ;;
val double : int -> int = <fun>

# let f = compose1 double add_one ;;
val f : int -> int = <fun>
# let result = f 3 ;;
val result : int = 8

# let f = compose2 double add_one ;;
val f : int -> int = <fun>
# let result = f 3 ;;
val result : int = 8
```

4 练习题4：验证列表符合函数要求

```
# let rec for_all func lst =
  match lst with
  | [] -> true
  | head :: tail ->
    if func head = true then for_all func tail
    else false ;;
val for_all : ('a -> bool) -> 'a list -> bool = <fun>
# let is_positive x = x > 0 ;;
val is_positive : int -> bool = <fun>
# let is_even x = x mod 2 = 0 ;;
val is_even : int -> bool = <fun>
# let r1 = for_all is_positive [1; 2; 3] ;;
val r1 : bool = true
# let r2 = for_all is_even [2; 4; 7] ;;
val r2 : bool = false
# let r3 = for_all is_positive [] ;;
val r3 : bool = true
```